

SolarWinds - Developer Guide

AEM Edge Delivery Services Repository Handover

Date: October 6, 2026

Table of Contents

SolarWinds - Developer Guide	3
Quick Reference	3
Architecture Overview	3
Local Development	5
Design System	6
Blocks Reference	7
Templates and Authoring Models	9
Common Development Tasks	9
Environments and Git Workflow	10
Troubleshooting	10
Resources and Support	11

SolarWinds - Developer Guide

Quick Reference

Resource	URL or value
Code repository	adobedrago/solarwinds-eds
Preview	main preview
Live	production site
Document Authoring	DA site
Content source	Markup source
Local development	http://localhost:3000
Storybook	http://localhost:6006

Config Service: org adobedrago, site solarwinds-eds, code repository adobedrago/solarwinds-eds. This is one standard site, not a repoless or multi-site configuration. Its content source is <https://content.da.live/adobedrago/solarwinds-eds> (markup).

Architecture Overview

Stack and Delivery Model

- AEM Edge Delivery Services markup authored in Document Authoring (DA), with vanilla JavaScript ES modules and CSS.
- There is no site-code compilation or bundling step. The platform serves repository files directly; the npm development dependencies support linting, design-token generation, and Storybook.
- The npm package manifest does not pin a Node.js version. Use a currently supported Node.js release compatible with the lockfile and dependencies.
- The generated design-token outputs are checked and rebuilt by scripts, but this is not a build step for the delivered site.
- Code and content have separate release paths: merging code to main ships code, while content is previewed and published separately from DA.

Repository Map

blocks/	31 block folders: shared blocks and SolarWinds patterns
cards-*/	Product, pricing, resource, and blog card patterns
columns-*/	Awards, event, logo, podcast, statistics, topic patterns
hero-*/	Homepage, blog, and contact-call-to-action heroes
tabs-*/	Deployment and solution tab patterns
header/ footer/	Site navigation and footer fragment decoration
fragment/	Loads and decorates authored page fragments
section-metadata/	Applies section style classes and data attributes
widget/	Resolves a widget link to HTML, CSS, and JavaScript
design/	
tokens/	Canonical Tokens Studio dictionaries and metadata

<code>tokens.css</code>	Generated CSS custom properties
<code>build-tokens.js</code>	Generator and consistency check entry point
<code>scripts/</code>	
<code>aem.js</code>	Vendored Edge Delivery Services core library; do not edit
<code>scripts.js</code>	Page loading, locale, auto-blocking, and decoration
<code>design-tokens.js</code>	Generated JavaScript breakpoint exports
<code>consent-check.js</code>	Delayed consent-state initialization
<code>consented.js</code>	Consent-gated integration entry point (currently a stub)
<code>placeholders.js</code>	Localized block labels and placeholder loading
<code>styles/</code>	
<code>styles.css</code>	Global styles, page-level sections, and theme aliases
<code>fonts.css</code>	Self-hosted font-face definitions
<code>lazy-styles.css</code>	Non-critical styles loaded after eager rendering
<code>storybook/</code>	HTML/Vite component catalog, fixtures, and stories
<code>fonts/</code>	Self-hosted WOFF2 brand fonts
<code>icons/ images/</code>	Shared icons and authored site assets
<code>tools/importer/</code>	Import tooling

There is no `templates/`, `models/`, `scripts/utils.js`, or `scripts/delayed.js` directory/file. The project uses DA authoring and block-local implementations rather than custom Universal Editor model JSON or custom template modules. The `blog` page metadata is still significant: core template decoration applies a `blog` body class, which selects blog-specific layout and footer behavior.

Page Loading and Decoration

`scripts/scripts.js` owns the page entry point and runs the Eager-Lazy-Delayed phases in sequence:

Phase	Repository behavior
Eager	Sets the document language from the URL; decorates core template/theme state and the main content; builds fragment and widget auto-blocks; decorates sections, blocks, and authored buttons; reveals the page and loads the first section. Font CSS may also be requested at desktop widths or when the session flag is present.
Lazy	Starts header and footer fragment loading, loads the remaining sections, restores a URL hash target, and loads <code>lazy-styles.css</code> and font CSS.
Delayed	Dynamically imports <code>consent-check.js</code> after eager and lazy work. It dispatches <code>consent.update</code> ; consented integrations belong in <code>consented.js</code> .

`buildAutoBlocks` transforms qualifying authored links before block decorators run. Links to `/fragments/` are loaded and decorated as fragments; links under `/widgets/` are converted to widget blocks. Treat backend markup as input data: inspect it first, and make decorators tolerant of authors adding, omitting, or reordering cells. The page entry also applies a Trusted Types policy that removes `srcdoc` attributes and embedded scripts at the relevant HTML parsing sinks.

`scripts/aem.js` is vendored core infrastructure and is explicitly protected by `AGENTS.md`: never edit it. `scripts.js` and blocks import core helpers such as `loadHeader`, `loadFooter`, `decorateIcons`, `decorateSections`, `decorateBlocks`, `decorateTemplateAndTheme`, `loadSection`, `loadSections`, `loadCSS`, `buildBlock`, and `picture` helpers from it. `blocks/fragment/fragment.js` is the one permitted cross-block import pattern; otherwise, shared code belongs under `/scripts/`.

Content and CSS Conventions

- Start block investigation with the backend-delivered markup: `curl -fsS http://localhost:3000/<path>.plain.html` (replace the path with the page route). Compare this source with the decorated browser DOM.

- Block classes are derived from the authored block name. Scope block CSS to `.blockname`; `-wrapper` and `-container` classes belong to the section shell and can be used for page composition.
- `section-metadata` consumes authored key/value rows. Its `style` key adds normalized classes to the section; other keys become `data-*` attributes. The metadata block removes itself after applying them.
- Use actual authored formatting and structure as the content contract. Do not assume fixed row or cell counts unless the block explicitly requires them.
- All committed files are served. Keep local-only or non-delivered artifacts excluded with `.hlxignore` as appropriate.

Local Development

Prerequisites and Startup

1. Install a supported Node.js release; no `.nvmrc` or `engines` field currently pins one.
2. Install the locked development dependencies with `npm ci`.
3. Run the standard single-site AEM proxy from the repository root:

```
npx -y @adobe/aem-cli up
```

Open `http://localhost:3000`. This workflow serves local code against previewed site content; no `--pages-url` override is needed for this standard single-site setup. Before changing a block, fetch its route's `.plain.html` response and confirm the authored structure.

Storybook and Quality Checks

```
npm run storybook
npm run storybook:build
npm run lint
```

The development catalog runs at `http://localhost:6006`. Its HTML/Vite stories reuse project fonts, icons, tokens, styles, fixtures, and block decorators. Stories provide both decorated renderings and undecorated backend-style rows/cells; shared controls update both forms. Header, footer, and fragment stories use static fixtures because their production behavior loads fragments and page-level behavior. `storybook:build` builds the catalog into `storybook-static`; it does not build the website.

`npm run lint` runs `tokens:check`, ESLint over the repository, and Stylelint over block, global, design, and Storybook CSS. There is no test or site build npm script. `npm run lint:fix` is available for lint autofixes; review its changes before keeping them.

Design Token Changes

`design/tokens/` is the canonical Tokens Studio dictionary. Do not edit the generated `design/tokens.css` or `scripts/design-tokens.js` by hand. After editing token source files, run:

```
npm run tokens:build
npm run tokens:check
npm run lint
```

Project-owned CSS should consume the generated custom properties instead of repeating design values. JavaScript breakpoint decisions should use `BREAKPOINTS` and `minWidth()` from `scripts/design-tokens.js`.

Native CSS media conditions cannot reference custom properties, so CSS breakpoints remain literals and must match the token dictionary.

Design System

Typography and Color

Category	Current system
Body and headings	Roboto variable font, weights 100-900; fallback uses local Arial with adjusted metrics.
Display	Roboto Slab variable font, weights 100-900.
Buttons	Geist variable font, weights 100-900, with Roboto fallback.
Additional face	Roboto Condensed Bold is defined in <code>styles/fonts.css</code> .
Brand primary	Orange #ff6200; pressed orange #ff9b66.
Brand secondary and links	Teal #006482; pressed teal #003c4e; subtle teal #d8e3e3.
Neutrals	White background; default text #111; body text #444; subtle background #f8f8f8; neutral surfaces include #eee and #f4f7f7.

The five self-hosted font files are `roboto-variable.woff2`, `roboto-italic-variable.woff2`, `roboto-slab-variable.woff2`, `geist-variable.woff2`, and `roboto-condensed-bold.woff2`. `fonts.css` uses `font-display: swap`; `scripts.js` loads font CSS outside the initial critical stylesheet.

Layout and Spacing

- Main content maximum width: 1280px; standard horizontal content padding: 24px.
- Section vertical padding: 48px on mobile and 64px at 600px and wider.
- Global spacing, sizing, radii, border widths, motion, opacity, and effects are tokenized alongside typography and color. Prefer these tokens over block-specific raw values.
- Global section styles include `light`, `highlight`, `light-teal`, `light-grey`, `dark`, and `teal`. Page-specific layouts such as `blog-hero`, `spotlight`, `panel`, and `catalog` containers are composed in global or owning-block CSS.

Breakpoints

The source includes these named breakpoints; page and block styles may also define content-specific thresholds between them.

Width	Common use
324px, 443px	Header banner line-wrap height corrections.
600px	Small/tablet threshold and switch to desktop section padding.
640px	Wider-mobile layouts and several card/hero layout changes.
768px	Tablet content rearrangement, including product modal highlights.
900px	Font-loading threshold and larger global type sizing.
963px, 993px	Header banner wrapping and blog navigation/layout changes.

Width	Common use
1024px	Desktop navigation and several block/sidebar layouts.
1104px	Header banner wrapping.
1200px, 1280px	Wide layouts and content maximum width.

The generated JavaScript exports `fonts: 900`, `blogNav: 993`, and `desktop: 1024`; it also provides `minWidth(width)`. Use the canonical token values for additional runtime decisions.

Blocks Reference

There are 31 maintained block folders. Specialized folders are distinct block names rather than generic variants. Where a README says “no variations,” presentation is driven by authored content or section classes instead. Most block modules transform DA rows/cells into semantic and accessible DOM and load their paired CSS when the block is discovered.

Block	Purpose and authored content	Behavior and variation notes
<code>cards</code>	General card rows with optional image and body cells.	Converts rows to a list, classifies image/body cells, and optimizes authored pictures.
<code>cards-authors</code>	Featured blog-author portraits, names, and job titles.	Whole card follows the author link; purpose-specific pattern, no documented class variation.
<code>cards-catalog</code>	Product catalog: category rows, product cards, type data, optional overview cells.	Builds category/type filters, result counts, responsive filter drawer, and accessible overview dialogs; media and video are deferred until opening. No class variations.
<code>cards-feature</code>	Three resource teasers authored as heading, description, and CTA rows.	Purpose-specific resource pattern; no class variations.
<code>cards-posts</code>	Blog teasers with linked image/title, optional category pills, date, and author.	Section styles change layout; <code>blog-hero</code> and <code>spotlight</code> have distinct card treatments.
<code>cards-pricing</code>	Pricing tiers authored as one card row per tier.	Text-led pricing cards; no documented class variations.
<code>cards-products</code>	Product pricing/list cards with icon, linked product name, pricing/details, and CTA cells.	Dedicated product list treatment; follow its README for the expected four-cell authoring contract.
<code>cards-resources</code>	Resource bento cards with image, eyebrow, linked title, and supporting copy.	First card is featured/wide; remaining cards follow the grid.
<code>cards-tools</code>	Three-column software/tool pricing grid with icon, name, and pricing content.	Dedicated tool pricing layout; section shell controls page-level composition.
<code>carousel</code>	Rows become slides with image/content columns.	Adds labelled controls and indicators for multiple slides; updates accessible slide/link state as slides change.
<code>carousel-testimonial</code>	Company logo, quote, CTA, person, role, and company rows.	Four testimonial entries on the source pattern; dedicated testimonial styling.
<code>columns</code>	General multi-column content rows.	Base columns pattern; use a named <code>columns-*</code> block when the content has a specific contract.

Block	Purpose and authored content	Behavior and variation notes
columns-awards	Award heading and badge images.	Two-cell award strip; responsive presentation.
columns-callout	Blog event promotion with heading, copy, CTA, and linked image.	Dedicated event-callout section design; no documented class variations.
columns-event	French homepage event promotion with optional linked image, copy, and CTA.	Full-bleed orange band and decorative accents are code-rendered, not authored; stacks below desktop widths.
columns-logos	Trust heading and customer-logo images.	Dedicated customer-logo row; section context can alter its presentation.
columns-podcast	Blog podcast heading and Wistia episode link.	Builds the podcast player from authored episode data; blog section styling owns placement.
columns-stats	Three statistics with highlighted number and descriptor.	Dedicated metrics row; includes French-specific styling.
columns-topics	Featured blog topic links and post metadata.	Three topic columns; entries are authored as linked title/date/author content.
footer	Loads footer content from a metadata override or locale-specific footer fragment.	Classifies authored brand, link, social, legal, and copyright sections; blog pages get a separately decorated footer.
fragment	Includes a DA fragment by path.	Fetches <code>.plain.html</code> , resolves fragment-relative media, decorates its sections, and replaces or flattens the wrapper.
header	Loads and decorates the navigation fragment.	Builds desktop mega menus and a keyboard-accessible mobile/tablet drill-down; locale labels and breakpoints come from runtime locale/tokens.
hero	Shared base hero block name.	The source JavaScript file is empty; concrete layouts live in <code>hero-banner</code> , <code>hero-blog</code> , and <code>hero-split</code> .
hero-banner	Contact CTA with optional decorative collage and heading/link row.	Content determines image/no-image layout; no author-supplied variation class is required.
hero-blog	Featured blog story with linked image, headline, date, author, and category.	Full-width stacked layout through 992px; two-column layout from 993px.
hero-split	Homepage hero with dashboard image, H1, supporting copy, and CTA.	Dedicated split hero layout; no documented class variations.
section-metadata	Authored section style and key/value metadata rows.	Adds normalized section classes or <code>data-*</code> attributes, then removes its own block markup.
tabs	Rows become tab panels with a tablist.	Adds tab roles and selection state; the shared base behavior does not implement arrow-key navigation.
tabs-deployment	Deployment choices and associated panel content.	Purpose-specific deployment filter/tabs; no documented class variations.
tabs-solutions	Solution labels and panels with image, copy, CTA, capabilities, and optional Wistia URL.	Four authored tabs enable the desktop interactive pie; video iframes load lazily on visibility/selection.
widget	Authored <code>/widgets/...</code> links resolve to widget assets.	Loads matching HTML, CSS, and JavaScript from <code>/widgets/</code> ; the widget decorator receives the

Block	Purpose and authored content	Behavior and variation notes
		loaded markup and query parameters as data attributes.

Block Development Rules

1. Read the block implementation, paired CSS, README/metadata, and a real `.plain.html` example before changing behavior.
2. Preserve an author-friendly DA table model. Be defensive about missing cells and rows unless the block's contract requires them.
3. The page-level auto-blocking pass runs before the block decorator, so inspect the DOM after that transformation when a link seems to disappear or change form.
4. Keep styles under the block class. Apply page-level section rules to section classes, not `-wrapper/-container` as if they were authored blocks.
5. Reuse `/scripts/` helpers and the one allowed fragment cross-block import pattern. Do not edit `scripts/aem.js`.
6. Add or update Storybook fixtures and stories for visible block changes. Verify both authored source structure and decorated output.

Templates and Authoring Models

There is no custom `templates/` folder or Universal Editor `models/` directory. The project is configured as a DA markup site. Blocks with a `README.md` document their authored table contract; some block directories also have `metadata.json` describing purpose and content patterns. Global page metadata is decorated by the core library. The `blog` template value is used by existing page behavior to add blog-specific layout/footer treatment, not by a project-owned template module.

Common Development Tasks

Add or Change a Block

1. Inspect a real page's `.plain.html` response and identify the authored rows/cells.
2. Create or update `blocks/<name>/<name>.js` and `blocks/<name>/<name>.css`; implement the default `decorate(block)` export expected by the block loader.
3. Keep DOM handling tolerant of optional author content and preserve semantic/accessibility state.
4. Add a representative fixture/story and verify decorated and undecorated views in Storybook.
5. Run `npm run lint` and test the page with the local AEM CLI.

Update Global Styling or Tokens

Use existing generated variables and scope styles to the correct block or section. For token changes, edit the source in `design/tokens/`, then regenerate, check, and lint. For CSS breakpoint changes, update the matching source token as well as the literal media condition.

Add a Consent-Gated Integration

Keep optional analytics or marketing code out of the eager page entry. The delayed path imports `consent-check.js`; consented code belongs in `scripts/consented.js` after the real consent integration is implemented. The current consent checker is a stub that declines by default. Use `?consent=accept` to exercise

its accepted path locally; `?consent=decline` exercises the default path. Do not mistake this test mechanism for a production consent-management platform.

Test a Content Change

Code preview and content preview are separate. DA content must be previewed/published from its authoring workflow. A code change is tested on its branch preview. Every pull request must include a branch preview URL and page path, following the requirement in `AGENTS.md` (for example, `https://<branch>--solarwinds-eds--adobedrago.aem.page/<path>`).

Environments and Git Workflow

Environment	URL pattern	Use
Local code	<code>http://localhost:3000</code>	AEM CLI proxy for local code and previewed content.
Branch preview	<code>https://<branch>--solarwinds-eds--adobedrago.aem.page/<path></code>	Verify the code change with the target route; include this URL in every pull request.
Main preview	<code>https://main-solarwinds-eds-adobedrago.aem.page/</code>	Preview current main-branch code.
Live	<code>https://main-solarwinds-eds-adobedrago.aem.live/</code>	Published production site.
DA	<code>https://da.live/#/adobedrago/solarwinds-eds/</code>	Author, preview, and publish markup content.

Keep feature branches focused and include the tested branch-preview route in the pull request. Merging to `main` deploys code; it does not publish DA content. Do not use retired `fstab.yaml`, `helix-query.yaml`, or `paths.json` files as active configuration; site configuration is managed by Config Service at `tools.aem.live`.

Troubleshooting

Symptom	Checks
Block does not load	Inspect the route's <code>.plain.html</code> ; confirm the authored block name/class and the generated wrapper; check the block JavaScript request and console for errors. Remember auto-blocking can rewrite widget and fragment links before decoration.
Block renders incorrectly	Compare backend rows/cells with the decorator's assumptions; test omitted/reordered cells; check whether <code>section-metadata</code> or a <code>-wrapper/-container</code> class controls page composition.

Symptom	Checks
CSS is missing or wrong	Check the block CSS network request, syntax/lint output, class scope, cascade, and breakpoint. Use token values; verify CSS media literals still match canonical tokens.
Header or footer is empty	Check the DA fragment path, locale prefix, and content preview status. A footer metadata value overrides the locale default; blog pages use their own footer behavior.
Wistia media is blank	Confirm the authored URL format and network access; some videos are intentionally created only when their tab/modal is selected or becomes visible.
Consent-gated code does not run	The current checker declines by default and the consented entry point is a stub. Exercise the local accepted path with <code>?consent=accept</code> ; confirm a real consent integration before adding production scripts.
Content appears stale	Confirm the DA page was previewed/published, then check the correct branch/main URL and browser/CDN cache. Content publishing and code merging are separate operations.

Resources and Support

Resource	URL
Edge Delivery Services documentation	https://www.aem.live/docs/
Developer tutorial	https://www.aem.live/developer/tutorial
Project anatomy	https://www.aem.live/developer/anatomy-of-a-project
Markup, sections, blocks, and auto-blocking	https://www.aem.live/developer/markup-sections-blocks
Performance guidance	https://www.aem.live/developer/keeping-it-100
Developer collaboration practices	https://www.aem.live/docs/dev-collab-and-good-practices
Block Collection	https://www.aem.live/developer/block-collection

No project-specific support contacts are recorded in the repository. Start with `AGENTS.md`, the relevant block README, and the DA source markup when investigating a change.